CONFIDENTIAL SECURITY ASSESSMENT

Swaptics - Updated Working Tree

Security, database, authentication and release-readiness audit

SECURITY SCORE	PRODUCTION READINESS	OVERALL RISK
2 / 10	**1 / 10**	**CRITICAL**

Branch: main | HEAD: 7fd3abf | Working tree contains staged changes and an unresolved merge

Prepared from static analysis plus an external-sandbox production build, test and lint verification.

Assessment date: 9 August 2026

1. Executive Summary

The updated main working tree must not be deployed. Although the branch pointer matches origin/main, the local tree contains staged changes and an unresolved merge conflict. The production build was rerun outside the sandbox and failed on merge markers and a duplicate ProductCarousel declaration.

The central security issue remains an incomplete Firebase-to-Supabase identity bridge: Firebase authenticates the UI, while ordinary Supabase requests remain anonymous. The anonymous admin bootstrap, cross-project token acceptance, anonymous profile insertion and unsafe historical administrator grant remain present.

The staged chat changes improve failure handling. However, the new authenticated-role admin RLS policies do not work for Firebase users while Supabase sees them as anon. Tests report one passing placeholder test and provide no meaningful security assurance. Lint reports 76 errors and 20 warnings.

Assessment Scope

- React/Vite routes and reusable UI components
- Firebase authentication and the Supabase profile bridge
- Supabase RLS policies, migrations, RPC functions and storage
- Edge Functions for profile creation, admin setup, user deletion and OTP
- Chat, listing, ratings, reports, notifications and admin workflows
- Vercel deployment headers, CSP, SEO, accessibility and performance
- Test coverage and operational readiness

Risk Summary

Severity	Count	Release impact
Critical	6	Immediate release blockers
High	8	Must remediate before launch
Medium / Quality	11	Required hardening and reliability work

2. Critical Findings

1. Anonymous first-admin takeover

CRITICAL

Abuse scenario: When the role table contains no administrator, any anonymous caller can provide a UUID and claim the first admin role.

Impact: Complete administrative takeover and access to user, content and moderation capabilities.

Evidence: supabase/migrations/20260716_fix_user_roles_bootstrap.sql defines bootstrap_admin(target_user_id), runs it as SECURITY DEFINER, and grants execution to anon.

Recommended fix: Revoke anonymous/public execution, remove browser bootstrapping and provision the first administrator through a controlled one-time deployment process.

2. Firebase identity is not authenticated to Supabase

CRITICAL

Impact: Listings, edits, profiles, chat, wishlists, ratings, reports, notifications and uploads are likely to fail because auth.uid() is NULL.

Evidence: AuthContext obtains a Firebase token only for firebase-profile. The shared Supabase browser client has no accessToken provider and uses the public key.

Recommended fix: Adopt one supported identity architecture: Supabase Auth; Supabase third-party Firebase Auth with token propagation; or verified Edge Functions for every authenticated operation.

3. Firebase tokens from other projects can be accepted

CRITICAL

Abuse scenario: The Google signing keys are shared infrastructure; signature validity alone does not bind the token to the Swaptics Firebase project.

Impact: A valid token from an attacker-controlled Firebase project may be treated as a Swaptics identity and may reach privileged workflows.

Evidence: firebase-profile and setup-admin verify signature and expiry but not alg, aud, iss, sub length, iat or auth_time.

Recommended fix: Use a maintained verifier and enforce RS256, the exact Firebase project audience, exact issuer, nonempty sub and all time claims.

4. Anonymous profile impersonation

CRITICAL

Impact: Untrusted callers can create arbitrary Firebase-linked profile records, causing impersonation, collisions and authorization confusion.

Evidence: 20260715_firebase_profile_bridge.sql permits INSERT when auth.uid() IS NULL and firebase_uid IS NOT NULL.

Recommended fix: Remove this exception. Service-role inserts already bypass RLS; all Firebase profile creation should happen only in the token-verified Edge Function.

5. Administrator granted by display-name substring

CRITICAL

Impact: The wrong profile can receive administrator privileges, including an attacker-controlled profile whose display name contains the selected substring.

Evidence: 20260716_grant_admin_srihari.sql searches profiles with full_name ILIKE '%srihari%' and grants admin to the earliest match.

Recommended fix: Remove the automated block. Provision administrators using an exact immutable Firebase UID verified out-of-band, with a one-time reviewed migration and a post-change audit.

6. Unresolved merge conflict blocks production build

CRITICAL

Impact: The application cannot be compiled or safely deployed from the current working tree.

Evidence: src/pages/Index.tsx contains conflict markers near lines 452, 474 and 496, and ProductCarousel is declared more than once. Vite confirmed both failures outside the sandbox.

Recommended fix: Resolve the merge deliberately, keep one ProductCarousel implementation, rerun build, lint and meaningful tests, then review the resulting diff before commit.

3. High-Risk Findings

7. Public disclosure of administrator identities

HIGH

Impact: Administrator user IDs are exposed and can be targeted.

Evidence: The Public can read user_roles policy uses USING (true).

Recommended fix: Remove public role-table access. Return only the minimum authorization result from trusted server code.

8. Client-side admin protection is not authorization

HIGH

Impact: Direct API calls can bypass hidden routes and UI checks.

Evidence: ProtectedRoute adminOnly redirects in React but does not secure backend resources.

Recommended fix: Authorize every admin mutation server-side after verifying the caller token and current role.

9. Competing profile-creation paths

HIGH

Impact: Race conditions, duplicate profiles and inconsistent identifiers.

Evidence: Auth.tsx inserts a Firebase UID directly while AuthContext calls an Edge Function that creates a random UUID.

Recommended fix: Delete the browser insert and keep one idempotent, verified profile resolution function.

10. User foreign keys conflict with Firebase identities

HIGH

Impact: Core writes may fail with foreign-key violations even after RLS is corrected.

Evidence: The bridge removes auth.users foreign keys only for profiles and user_roles; products, chats, messages, ratings and reports still reference auth.users.

Recommended fix: Use actual Supabase identities or redesign every user reference around a dedicated application-user table.

11. Administrative deletion is fragmented and non-atomic

HIGH

Impact: Partial deletion can leave private or orphaned data and mismatched Firebase/Supabase accounts.

Evidence: Admin.tsx performs a long sequence of client-side deletes.

Recommended fix: Use one verified server transaction, cascading relationships, Firebase Admin deletion and immutable audit logs.

12. OTP and email endpoint abuse

HIGH

Impact: Attackers can generate outbound email cost, enumerate accounts and brute-force weak verification flows.

Evidence: Rate limiting is email-only; no IP/device/global throttle or CAPTCHA is present. The OTP workflow also conflicts with Firebase signup.

Recommended fix: Remove the obsolete flow or add layered rate limits, CAPTCHA, generic responses, atomic counters and HMAC-based OTP storage.

13. Unsafe file-upload trust boundary

HIGH

Impact: Malicious or oversized files can consume storage, bypass expected formats or create unsafe content.

Evidence: Sell.tsx trusts the browser accept attribute, filename extension and public object URL.

Recommended fix: Quarantine uploads; verify signatures, size and dimensions; strip metadata; re-encode images; enforce quotas; and clean up failed uploads.

14. Unrestricted setup-admin bootstrap

HIGH

Impact: Any Firebase-authenticated caller can become the first administrator, and the action=fix path can create privileged Supabase users during bootstrap.

Evidence: setup-admin treats zero existing admins as sufficient authorization before privileged account operations.

Recommended fix: Remove runtime bootstrap. Require a one-time exact UID migration, then require an existing administrator for every subsequent action.

4. Medium-Risk, Reliability and Product Findings

15. Weak CSP

The policy permits unsafe-inline and unsafe-eval. Replace inline scripts with nonces/hashes and remove unsafe-eval.

16. Missing operational controls

Add audit logging, role-change alerts, key rotation, incident response, session revocation and restore drills.

17. Silent chat failures

Message and notification insert errors are discarded. Await inserts, use server IDs and offer retry states.

18. Non-transactional listing edits

Failed database writes can leave orphaned uploads; removed images are not deleted; object URLs are not revoked.

19. Incomplete business validation

Enforce positive prices, dates, lengths, enumerations, image counts and buyer/seller rules in the database.

20. Accessibility gaps

Add names to icon buttons, status semantics, focus handling, a skip link, keyboard testing and accessible destructive confirmations.

21. SEO gaps

SPA-only metadata may not reach crawlers. Add prerendering/SSR, structured data, dynamic sitemaps and default social images.

22. Performance risks

Add pagination, narrow selects, eliminate N+1 profile queries, transform images and simplify duplicate realtime paths.

23. Insufficient testing

Only a placeholder test exists. Add RLS, auth, ownership, admin, upload, chat, accessibility and end-to-end tests.

24. Quality gate currently fails

Lint reports 76 errors and 20 warnings, including widespread any types, ignored TypeScript errors and hook dependency problems. Make lint and tests mandatory in CI.

25. New admin RLS policies are ineffective for Firebase sessions

The staged 20260808_fix_admin_rls_policies.sql grants permissions to authenticated and checks auth.uid(), but the browser Supabase client still operates as anon. Fix the identity bridge before relying on these policies.

5. Required Access to Implement All Fixes

Use named individual accounts, MFA and least privilege. Do not share passwords, recovery codes or service-role keys through chat. Grant temporary access where possible and revoke it after remediation.

System	Minimum practical access	Why it is needed
Source repository	Write access to a remediation branch; permission to open PRs and run CI	Frontend, Edge Function, migration, tests and configuration changes
Supabase project	Developer for normal work; Owner/Admin only for secrets, auth providers and production recovery	RLS, schema migrations, functions, storage, logs, backups and auth configuration
Supabase database	Migration runner/service account; read-only production inspection first	Apply and verify policies, constraints, RPC revocation and user-key redesign
Firebase project	Firebase Authentication Admin plus project Viewer; temporary higher access only if provider/domain configuration requires it	Authorized domains, identity providers, token issuer/audience, users and Admin SDK integration
Vercel	Project Developer; Project Admin for environment variables/domains	Preview deployments, headers, CSP, secrets, logs and rollback
DNS/domain	DNS editor or supervised change window	Firebase authorized domain, verification records, redirects and security records
Email provider	Developer/API-key manager with restricted sending-domain scope	OTP removal/hardening, sender domain and abuse monitoring
Analytics	Read access initially; Editor only for consent and configuration fixes	Privacy review, event validation and unnecessary data collection
Monitoring/logging	Read and alert-management access	Detect abuse, privilege changes, failures and suspicious authentication

Information and approvals also required

- A staging environment that mirrors production without real customer data
- Current production database schema and migration history
- Inventory of deployed Edge Functions and their environment variable names
- Firebase project ID and approved identity architecture decision
- Backup and tested rollback approval before database identity migrations
- A designated person authorized to approve the first administrator
- Test accounts for anonymous, user, seller, buyer and administrator roles
- Permission for an authorized staging penetration test after remediation

6. Secure Access Handover Procedure

- Invite the engineer by their own email account; never share your owner account.
- Require MFA for Git hosting, Supabase, Firebase/Google Cloud, Vercel and DNS.
- Start with read-only inspection, then grant scoped write access for the active remediation phase.
- Keep production service-role keys and Firebase service-account keys in platform secret stores only.
- Use preview/staging deployments for all changes and require reviewed pull requests.
- Take and verify a database backup before identity or foreign-key migrations.
- Record every production change, migration ID, approver and rollback command.
- Revoke temporary roles and rotate exposed or broadly shared keys after completion.

Access that should not be handed over directly

- Personal passwords or MFA recovery codes
- Unrestricted service-role keys in email or messaging apps
- Registrar ownership when a scoped DNS collaborator role is available
- Production customer-data exports
- Permanent organization owner access unless operationally necessary

7. Prioritized Remediation Plan

Phase 0 - Containment (same day)

- Revoke and remove anonymous admin bootstrap
- Disable unsafe profile insertion
- Restrict role-table reads
- Confirm whether any unauthorized administrators or profiles already exist
- Rotate sensitive credentials if exposure is suspected

Phase 1 - Identity redesign (highest priority)

- Choose the supported Firebase/Supabase model
- Validate issuer, audience and all token claims
- Unify profile creation and identifiers
- Reconcile foreign keys and RLS
- Add automated cross-user and anonymous-denial tests

Phase 2 - Privileged and transactional operations

- Move admin mutations to verified server code
- Implement atomic account deletion
- Secure uploads
- Remove or harden OTP/email endpoints
- Add audit logging and alerts

Phase 3 - Product reliability

- Handle chat and mutation failures

- Add database constraints and cleanup
- Paginate admin screens
- Remove N+1 queries
- Build meaningful integration and end-to-end tests

Phase 4 - Launch validation

- Tighten CSP
- Complete accessibility and SEO work
- Run dependency and secret scans
- Test backups and rollback
- Perform an authorized staging penetration test

8. Release Gate

Production release should remain blocked until all critical findings and the identity-model issues are resolved and verified with automated authorization tests.

Minimum evidence for approval

- Anonymous callers cannot create profiles, roles or privileged records
- Tokens from any other Firebase project are rejected
- Every protected Supabase request carries a verifiable user identity
- A user cannot read or mutate another user's private resources
- Admin access is granted only through an approved, audited process
- Identity and foreign-key migrations pass on a restored staging backup
- Critical user journeys pass end-to-end tests
- Security headers, upload controls, rate limits, monitoring and rollback are verified

Conclusion

The application has a solid modern frontend foundation and some useful security headers, but its trust boundary is currently inconsistent. Fixing the authentication architecture first will make the remaining database, admin, upload and product hardening work substantially simpler and safer.

End of report